SDE-based Modelling and Calibration of Indian Equity Prices

From SDEs to Fokker–Planck PDEs and PINN/Neural-ODE based Inference

1. Introduction

This project will develop, calibrate, and validate stochastic differential equation (SDE) models for Indian equity prices, focusing on the NIFTY 50 index and selected large-cap stocks. The work couples classical SDE modelling (GBM, Heston, jump-diffusion) with course-relevant deterministic partial differential equations (Fokker–Planck and Black–Scholes) and modern machine learning approaches, specifically Neural Ordinary Differential Equations (Neural ODEs) and Physics-Informed Neural Networks (PINNs). The core aim is to achieve: (a) rigorous mathematical derivation and robust implementation of these models, (b) a data-driven comparison of classical versus ML-based inference techniques, and (c) the extraction and backtesting of key risk measures (VaR/ES) while verifying stylized-fact replication. This project not only pursues a relevant quantitative finance research direction but also explicitly integrates and applies core topics from the MA-311 curriculum.

2. Motivation & Objectives

Layman's View

Financial markets are inherently noisy and unpredictable. Our goal is to use principled mathematical models that can capture this randomness, including sudden volatility spikes and market crashes. We will build these models, fit them to real Indian market data, and leverage modern AI techniques to infer the hidden dynamics driving asset prices.

Technical Objectives

1. **Formulate and Implement SDE Models:** Develop robust code for baseline SDEs, including Geometric Brownian Motion (GBM), Heston (stochastic volatility), Merton (jump-diffusion), and a hybrid model combining stochastic volatility with jumps.
2. **Derive Associated PDEs:** Formally derive the Fokker–Planck (Kolmogorov forward) PDE for the probability density of each SDE and establish the Black–Scholes PDE link for option pricing.

3. **Implement Classical Calibration:** Code and apply classical parameter estimation methods, such as Maximum Likelihood Estimation (MLE), moment matching, and where necessary, filtering techniques like EKF or Particle Filters for latent state models.

4. **Implement ML-based Inference:** Develop ML-driven estimation pipelines: (a) a Neural ODE to learn the drift component from time-series data, and (b) a PINN to solve the inverse problem for the Fokker–Planck PDE, inferring its coefficients from empirical data distributions.

5. **Perform Rigorous Evaluation:** Assess all models based on goodness-of-fit, replication of stylized facts (e.g., volatility clustering, heavy tails), forecasting accuracy, and the performance of risk metrics (VaR/ES) via formal backtesting procedures.

6. **Produce Reproducible Deliverables:** Create a well-documented codebase, compelling visualizations, and a final research report suitable for academic submission.

3. Relevance to MA-311 Course Modules

This project is designed to directly apply concepts from across the MA-311 syllabus:

- **Module 1 (Intro / Model fitting):** Central themes include model selection (e.g., GBM vs. Heston), various model fitting techniques (MLE, calibration), and using simulation to validate model behavior.
- **Module 2 (Systems of DEs):** Involves the derivation of the Fokker–Planck PDE from the underlying SDE, analyzing the deterministic limit of stochastic systems, and studying the stability of drift/mean fields.
- **Module 3 (ML & Neural ODEs / PINNs):** Directly applies advanced topics by framing model discovery as a PINN inverse problem to learn the diffusion term and using Neural ODEs to learn drift vector fields, all optimized via gradient-based methods.
- **Module 4 (Case study):** Option pricing via the Black–Scholes PDE and risk management via VaR backtesting serve as concrete, finance-oriented case studies. The PINN application is a direct extension of the course's examples.

4. Mathematical Models — Detailed Formulation

4.1 Geometric Brownian Motion (GBM) — Baseline

The SDE is given by $dS_t = \mu S_t\,dt + \sigma S_t\,dW_t$. This implies the log-price dynamics are an arithmetic Brownian motion:

$$d\ln S_t = \left(\mu - \tfrac{1}{2}\sigma^2\right)\,dt + \sigma\,dW_t.$$

Parameters μ and σ can be estimated in closed-form via MLE from historical log-returns.

4.2 Heston Stochastic Volatility Model

This two-factor model captures volatility clustering and mean reversion:

$$\begin{aligned} dS_t &= \mu S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^S, \\ dv_t &= \kappa(\theta - v_t)\,dt + \xi\sqrt{v_t}\,dW_t^v, \\ d\langle W^S, W^v\rangle_t &= \rho\,dt. \end{aligned}$$

Here, v_t is the stochastic variance, which mean-reverts to a long-term level θ at a rate κ. ξ is the volatility of volatility, and ρ is the correlation between asset and volatility shocks.

4.3 Jump-Diffusion (Merton)

To model market crashes and large, sudden movements, we add a compound Poisson process:

$$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t + S_{t^-}(Y-1)\,dN_t,$$

where N_t is a Poisson process with intensity λ and Y is an i.i.d. random variable representing the jump size multiplier.

4.4 Fokker–Planck PDE

For a general Itô process $dX_t = a(X_t,t)\,dt + b(X_t,t)\,dW_t$, the probability density function $p(x,t)$ of X_t evolves according to the Fokker-Planck (or Kolmogorov Forward) equation:

$$\frac{\partial p}{\partial t} = -\frac{\partial}{\partial x}[a(x,t)p(x,t)] + \frac{1}{2}\frac{\partial^2}{\partial x^2}[b(x,t)^2 p(x,t)].$$

For jump processes, this is extended to a partial integro-differential equation.

4.5 Black–Scholes PDE

Under risk-neutral dynamics for GBM and the no-arbitrage principle, the price $V(S,t)$ of a European derivative must satisfy the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} - rV = 0.$$

5. Numerical & Estimation Methods

5.1 Classical Methods

▷ **GBM:** MLE on log-returns (closed-form).

▷ **Heston:** Calibration via (i) moment-matching on realized volatility, (ii) maximum likelihood with a particle filter, or (iii) option-based calibration using FFT methods. We will focus on (i) and (ii).

▷ **Jump Models:** EM algorithm or likelihood methods with jump detection via thresholds.

▷ **Simulators:** Euler–Maruyama, Milstein, and specialized schemes like Andersen's QE scheme for Heston to ensure positivity of variance.

5.2 Neural ODE for Drift Inference

We parameterize the drift function $a_\theta(s,t)$ as a small neural network. Using discrete observations of the log-price, we train the network by minimizing the prediction loss after integrating the learned dynamics forward in time, using an adjoint-based solver like `torchdiffeq`.

5.3 PINN for Fokker-Planck Inverse Problem

We represent the unknown drift $a_\theta(s,t)$ and diffusion $b_\phi(s,t)$ as neural networks. The loss function is a weighted sum of two components:

1. **PDE Residual:** The L_2 norm of the Fokker-Planck PDE evaluated at random collocation points in the (s,t) domain. This enforces the physics.
2. **Data Mismatch:** The difference between the PINN's predicted density $p_\theta(s,t_i)$ and the empirically observed histogram density at different time slices t_i.

6. Data Acquisition and Preprocessing Plan

Proposed Assets

- **Index:** NIFTY 50 (`^NSEI`).
- **Large-Cap Stocks:** A selection of 3-4 liquid stocks such as Reliance Industries, TCS, and HDFC Bank.
- **Timeframe:** Daily OHLCV data from January 1, 2015 to present day.

Data Sources

- Primary: `yfinance` Python package for stock and index data.
- Secondary: NSE India official website for historical data verification.
- Auxiliary: Reserve Bank of India (RBI) for treasury bill rates as a risk-free proxy.

7. Timeline (12-Week Plan)

Weeks 1–2: Setup & Baseline. Finalize assets, download and clean data. Perform EDA and compute stylized facts. Fit GBM and GARCH baselines.

Weeks 3–5: Advanced SDEs. Implement simulators for Heston (QE scheme) and jump-diffusion models. Perform initial calibration using moment matching.

Weeks 6–8: PINN Development. Build empirical densities from data. Implement the PINN framework with a combined loss function (PDE residual + data mismatch).

Weeks 9–10: Neural ODE Development. Implement the Neural ODE drift learner using `torchdiffeq`. Train and validate on log-price data.

Weeks 11–12: Experiments & Analysis. Test all models, generate forecasts, and compute risk metrics. Backtest VaR/ES. Compare all models quantitatively.

Weeks 12: Finalization. Final report, presentation, and codebase for submission.

8. Expected Risks & Mitigation Strategies

Risk PINNs can be difficult to train and computationally expensive.

Mitigation Start with a simpler 1D PDE (e.g., on log-price), use a small network architecture, and potentially pre-train on synthetic data from a known SDE. Use transfer learning if applicable.

Risk Calibrating the Heston model can be unstable, especially without access to options data.

Mitigation Rely on moment matching with realized volatility as a robust proxy. Clearly state the limitations of this approach in the final report.

Risk The project scope is broad, posing a time constraint risk.

Mitigation Prioritize a core set of deliverables: a robust implementation of the GBM/Heston models and one of the two ML methods (e.g., Neural ODE). The second ML method can be treated as a stretch goal.

9. Key References

- Chen, R. T. Q., et al. (2018). Neural Ordinary Differential Equations. *NeurIPS*.
- Heston, S. L. (1993). A Closed-Form Solution for Options with Stochastic Volatility. *Review of Financial Studies*.
- Kloeden, P. E. & Platen, E. (1992). *Numerical Solution of Stochastic Differential Equations*.
- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-Informed Neural Networks. *Journal of Computational Physics*.